Image Analytics with Computer Vision – Lab Programs (AML23701)

Oct 22, 2026 · @Chethan

1. Gray, HSV and CMYK conversion

OpenCV converts to Gray and HSV directly; CMYK has no built-in code, so it is computed from RGB.

```
import cv2, numpy as np

img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

b, g, r = cv2.split(img / 255.0)
k = 1 - np.maximum(np.maximum(r, g), b)
c, m, y = [(1 - x - k) / (1 - k + 1e-8) for x in (r, g, b)]

for name, im in [('Gray', gray), ('HSV', hsv), ('C', c), ('M', m), ('Y', y),
('K', k)]:
    cv2.imshow(name, im)
cv2.waitKey(0)
```

2. Histogram equalization

The equalized histogram spreads across 0–255, which raises contrast.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg', 0)
eq = cv2.equalizeHist(img)

plt.hist(img.ravel(), 256, (0, 256)); plt.title('Original'); plt.figure()
plt.hist(eq.ravel(), 256, (0, 256)); plt.title('Equalized'); plt.show()
```

3. Morphological operations

The image is thresholded to binary, then four operations run with a 5×5 kernel.

```
import cv2, numpy as np

img = cv2.imread('image.jpg', 0)
_, b = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
k = np.ones((5, 5), np.uint8)

cv2.imshow('Dilation', cv2.dilate(b, k))
cv2.imshow('Erosion', cv2.erode(b, k))
cv2.imshow('Opening', cv2.morphologyEx(b, cv2.MORPH_OPEN, k))
cv2.imshow('Closing', cv2.morphologyEx(b, cv2.MORPH_CLOSE, k))
cv2.waitKey(0)
```

4. Fourier Transform with low-pass and high-pass filters

A circular mask of radius 30 keeps low frequencies (blur); its inverse keeps high frequencies (edges).

```
import cv2, numpy as np

img = cv2.imread('image.jpg', 0)
F = np.fft.fftshift(np.fft.fft2(img))
r, c = img.shape
y, x = np.ogrid[:r, :c]
low = ((y - r // 2) ** 2 + (x - c // 2) ** 2 <= 30 ** 2)

back = lambda m: np.abs(np.fft.ifft2(np.fft.ifftshift(F *
m))).astype(np.uint8)

cv2.imshow('Low-pass', back(low))
cv2.imshow('High-pass', back(~low))
cv2.waitKey(0)
```

5. Laplacian edge detection

A light Gaussian blur first reduces noise, since the Laplacian amplifies it.

```
import cv2

img = cv2.imread('image.jpg', 0)
lap = cv2.convertScaleAbs(cv2.Laplacian(img, cv2.CV_64F))

cv2.imshow('Laplacian', lap)
cv2.waitKey(0)
```

6. Hough Lines (rho, theta)

Canny edges vote in (rho, theta) space; each detected pair is converted back to two points to draw the line.

```
import cv2, numpy as np

img = cv2.imread('image.jpg')
edges = cv2.Canny(cv2.cvtColor(img, cv2.COLOR_BGR2GRAY), 50, 150)
lines = cv2.HoughLines(edges, 1, np.pi / 180, 150)

for rho, theta in lines[:, 0]:
    a, b = np.cos(theta), np.sin(theta)
    x0, y0 = a * rho, b * rho
    cv2.line(img, (int(x0 - 1000 * b), int(y0 + 1000 * a)),
             (int(x0 + 1000 * b), int(y0 - 1000 * a)), (0, 0, 255), 2)

cv2.imshow('Hough Lines', img)
cv2.waitKey(0)
```

7. YOLOv8

Install once with `pip install ultralytics`; the pretrained nano weights download on first run.

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
results = model('image.jpg')
results[0].show()
```

8. Faster R-CNN

Install once with `pip install torch torchvision`; detections above a 0.6 score are drawn.

```
import torch, torchvision, cv2
from torchvision.transforms.functional import to_tensor

model =
torchvision.models.detection.fasterrcnn_resnet50_fpn(weights='DEFAULT').eval(
)
img = cv2.imread('image.jpg')

with torch.no_grad():
    out = model([to_tensor(img[..., ::-1].copy())])[0]

for box, score in zip(out['boxes'], out['scores']):
    if score > 0.6:
        x1, y1, x2, y2 = map(int, box)
        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

cv2.imshow('Faster R-CNN', img)
cv2.waitKey(0)
```